Name: Manoj M
Reg no: 192511060
course Name: Data structures
course code: CSA0306

14/08/26

Sorting Algorithm

1. Insertion sort:

Insertion sort is a simple comparison-based sorting Algorithm that builds that builds the sorted array one element at a time by inserting each element into its correct position among the previously sorted elements.

concept

- Divide the array into sorted and unsorted portions
- Initially 1st element is sorted.
- take next element as key.
- compare key with elements on it left.
- shift larger elements one position right.
- insert key at its correct position.

Algorithm

```
InsertionSort (A, n)
start
FOR  i = 1  to  n-1
        key = A[i]
        j = i-1
        while j >= 0  AND  A[j] > key
                A[j+1] = A[j]
                j = j - 1
        A[j+1] = key
STOP
```

complexity

- Best case - $O(n)$
- Average case - $O(n^2)$
- worst case - $O(n^2)$

space complexity : $O(1)$

2. Merge sort:

Merge sort is a divide-and conquer sorting Algorithm that divides an array into smaller subarrays, sorts them recursively and then merges the sorted subarrays.

concept

- merge sort follows 3 main steps:

Divde → conquer → Merge

Algorithm

```
Merge sort (A, low, High)
If low < high
    mid = (low + high) / 2
    Mergesort (A, low, mid)
    Mergesort (A, mid+1, High)
     merge (A, low, mid, High)

Stop
```

complexity

Time :

Best - O(nlogn)

Average - O(nlogn)

worst - O(nlogn).

Space : O(n).

3) Radix sort

Radix sort is a non-comparison based sorting Algorithm that sorts numbers digit by digit, starting from the least significant digit (LSD)

concepts

170 45 75 90 802 24 2 66

sort according to units → tens → hundreds.

After units

170 90 802 2 24 45 75 66

After tens.

802 2 24 45 66 170 75 90

Algorithm

Radixsort (A, n)

Max element → Find no. of maximum digit

place = 1

while max/place > 0

sort According to digit

place = place x 10

STOP

complexity

time:

Best - $O(d(n+k))$

Average - $O(d(n+k))$

worst - $O(d(n+k))$

space: $O(n+k)$

4) Quick sort

Quick sort is a divide and conquer method comparison based algorithm that selects a pivot and partitions the array so that smaller elements are placed on one side.

concept

- choose pivot → partion → Recursively sort

Algorithm

```
quicksort ( A , low, high )

IF   low < high
        pivot = partion (A , low, high)
        quicksort (A , low, pivot-1)
        quicksort (A , pivot+1, high)

STOP
```

complexity

time: best - $O(n \log n)$

Average - $O(n \log n)$

worst - $O(n^2)$

space: $O(n)$.

5. Bubble sort

Bubble sort is a simple comparison based sorting algorithm that repeatedly compares adjacent element and swaps them if they are in the wrong order.

concept

for Ascending order

if $A[j] > A[j+1]$

=> swap them.

Algorithm

Bubble-sort (A, n)

START

FOR $i = 0$ to $n-1$

FOR $j = 0$ to $n-i-2$

IF $A[j] > A[j+1]$

SWAP $A[j]$ and $A[j+1]$

STOP

complexity

time : Best - $O(n)$

Avg - $O(n^2)$

worst - $O(n^2)$

space : $O(1)$

Comparison Table =>

Sorting Algo	Best case	Avg case	worst case	space complexity	Reason / intervention
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Best for small
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Given guaranteed if $n(\log n)$ performance
Radix sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	Efficient for digit
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$.	usually very fast in practice.
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	very simple, but inefficient for large data